

Phase 3: Resilience & Graceful Degradation Protocols

3.1 Database Fallback (RDS → Local SQLite)

The primary datastore target is resolved once, at process startup, from `app/core/config.py`. This is a **static configuration default**, not a runtime health-probe/failover — there is no code path that detects a slow or unreachable remote database mid-session and re-routes traffic. If the connection is going to fail, it fails at engine-creation time, not silently mid-request.

```
# app/core/config.py
import os
from pydantic_settings import BaseSettings
from pydantic import model_validator
from typing import Optional
from dotenv import load_dotenv
from urllib.parse import quote

load_dotenv()

class Settings(BaseSettings):
    PROJECT_NAME: str = "NL2SQL Pipeline"
    API_V1_STR: str = "/api/v1"

    # Database Settings (Target DB to query)
    DATABASE_URL: str = os.getenv("DATABASE_URL",
    "sqlite:///./derivinsightnew.db")
    SCHEMA_PATH: str = "app/files/derivinsight_schema.sql"
    MOCK_DATA_SCRIPT_PATH: str = "app/files/generate_mock_data.py"

    class Config:
        env_file = ".env"
        extra = "ignore"

settings = Settings()
```

Resolution logic, expressed explicitly:

1. `os.getenv("DATABASE_URL", "sqlite:///./derivinsightnew.db")` is evaluated once when `Settings()` is instantiated (module import time, i.e. process start).
2. If the `DATABASE_URL` environment variable is unset **or commented out** in `.env`, `os.getenv` returns `None` for the lookup and the second positional argument (`"sqlite:///./derivinsightnew.db"`) is used as the default.

3. If `DATABASE_URL` is set — e.g. to a remote MySQL/PostgreSQL DSN such as `mysql+pymysql://admin:<password>@<rds-host>:3306/derivinsight` — that value is used verbatim, with no reachability check at this layer.

The engine is then constructed downstream in `app/services/database.py`, with SQLite-specific connection arguments applied conditionally on a substring check of the resolved URL:

```
# app/services/database.py
import os
from typing import List, Dict, Any
from sqlalchemy import create_engine, text, inspect
from app.core.config import settings
from app.core.logger import logger

class DatabaseService:
    def __init__(self):
        connect_args = {"check_same_thread": False} if "sqlite" in
settings.DATABASE_URL else {}
        self.engine = create_engine(settings.DATABASE_URL,
connect_args=connect_args)
        logger.info(f"DatabaseService initialized with engine:
{settings.DATABASE_URL.split('///')[1]}")
        self.initialize_db()

    def initialize_db(self):
        """Initialize database schema if it doesn't exist."""
        try:
            inspector = inspect(self.engine)
            if not inspector.has_table("users"):
                print("Initializing database schema...")
                self._execute_sql_file(settings.SCHEMA_PATH)
                print("Schema initialized.")
        except Exception as e:
            print(f"Error initializing database: {e}")

    def _execute_sql_file(self, file_path: str):
        """Execute SQL commands from a file, one transaction, rollback
on any failure."""
        if not os.path.exists(file_path):
            file_path = os.path.join(os.getcwd(), file_path)
        if not os.path.exists(file_path):
            print(f"Schema file not found at: {file_path}")
            return

        with open(file_path, 'r', encoding='utf-8') as f:
            sql_content = f.read()

        statements = sql_content.split(';')
```

```

with self.engine.connect() as conn:
    try:
        with conn.begin():
            for statement in statements:
                if statement.strip():
                    conn.execute(text(statement))
    except Exception as e:
        print(f"Database error! Rolled back to prevent
corruption: {e}")

```

Auto-bootstrap behavior: on every process start, `initialize_db()` inspects the resolved database for a `users` table. If it's absent — the normal case for a fresh local SQLite file — it executes the full schema file at `settings.SCHEMA_PATH` (`app/files/derivinsight_schema.sql`) inside a single transaction, rolling back atomically if any statement fails. This means the local fallback path is not just "connect to a file" — it self-provisions its own schema with zero manual setup.

There are **two independent SQLite databases** in this system, each owned by a different subsystem and each with its own fallback default:

Database file	Owning subsystem	Config key
<code>derivinsightnew.db</code>	NL2SQL pipeline (<code>DatabaseService</code>)	<code>DATABASE_URL</code> in <code>app/core/config.py</code>
<code>derivinsight_alerts.db</code>	Alert Engine (<code>AlertEngineService</code>)	<code>ALERTS_DB_PATH</code> in <code>app/services/alert_engine.py</code>

Correction against reference context: the reference material for this document describes this as a "database fallback circuit" in the same breath as the Redis circuit breaker below. In source, these are architecturally different: the Redis path (§3.2) actively probes the connection at init and on every access, catching exceptions and demoting to `None`. The database path performs no such probe — it is a one-time environment-variable default substitution. Calling both a "circuit breaker" overstates the DB path's runtime behavior; it should be described as a **config-level fallback default**, not a breaker.

3.2 Redis Circuit Breaker — Worker Registry

`app/services/worker_registry.py` implements a defensive, exception-scoped circuit breaker around Redis connectivity for tracking ECS `taskArn` values for the engine and generator workers.

```
# app/services/worker_registry.py
"""
Worker Registry (Redis)
=====
Stores taskArn for engine and generator workers so the API can:
- Report status: if key present → worker is running
- Stop: fetch taskArn from Redis, stop the task, then remove the key
"""

from typing import Optional
import redis

from app.core.config import settings

REDIS_KEY_ENGINE_TASK_ARN = "alerts:engine:task_arn"
REDIS_KEY_GENERATOR_TASK_ARN = "alerts:generator:task_arn"

class WorkerRegistry:
    """Redis-backed registry for engine and generator worker task
    ARNs."""

    def __init__(self, redis_url: str = None):
        self.redis_url = redis_url or settings.REDIS_URL
        self._client: Optional[redis.Redis] = None
        self._available = False
        self._init_redis()

    def _init_redis(self):
        """Initialize Redis connection safely, falling back if None or
        unreachable."""
        if not self.redis_url:
            print("[WorkerRegistry] REDIS_URL is None. Operating in
            fallback mode.")
            self._client = None
            return

        try:
            self._client = redis.from_url(
                self.redis_url,
                decode_responses=True,
                socket_connect_timeout=3.0,
                socket_timeout=3.0
            )
            self._client.ping()
        except (redis.exceptions.TimeoutError,
```

```

redis.exceptions.ConnectionError, redis.RedisError) as e:
    print(f"[WorkerRegistry] Redis connection failed: {e}."
          "Operating in fallback mode.")
    self._client = None

@property
def redis(self) -> Optional[redis.Redis]:
    if self._client is None:
        self._init_redis()
    return self._client

def set_engine_task_arn(self, task_arn: str) -> bool:
    if not self.redis:
        return False
    try:
        self.redis.set(REDIS_KEY_ENGINE_TASK_ARN, task_arn)
        return True
    except redis.RedisError:
        return False

def get_engine_task_arn(self) -> Optional[str]:
    if not self.redis:
        return None
    try:
        return self.redis.get(REDIS_KEY_ENGINE_TASK_ARN)
    except redis.RedisError:
        return None

def delete_engine_task_arn(self) -> bool:
    if not self.redis:
        return False
    try:
        self.redis.delete(REDIS_KEY_ENGINE_TASK_ARN)
        return True
    except redis.RedisError:
        return False

def set_generator_task_arn(self, task_arn: str) -> bool:
    if not self.redis:
        return False
    try:
        self.redis.set(REDIS_KEY_GENERATOR_TASK_ARN, task_arn)
        return True
    except redis.RedisError:
        return False

def get_generator_task_arn(self) -> Optional[str]:
    if not self.redis:
        return None
    try:
        return self.redis.get(REDIS_KEY_GENERATOR_TASK_ARN)

```

```

except redis.RedisError:
    return None

def delete_generator_task_arn(self) -> bool:
    if not self.redis:
        return False
    try:
        self.redis.delete(REDIS_KEY_GENERATOR_TASK_ARN)
        return True
    except redis.RedisError:
        return False

def is_available(self) -> bool:
    return self._available

# Singleton for API use
worker_registry = WorkerRegistry()

```

Breaker mechanics, confirmed line-by-line against source:

- **Guard before connect:** `if not self.redis_url` short-circuits before any socket operation — this specifically prevents an `AttributeError` /malformed-URL crash from calling `redis.from_url(None, ...)`, which is a known failure mode when a `.env` var is present but empty.
- **Bounded timeouts:** `socket_connect_timeout=3.0` and `socket_timeout=3.0` — a hung Redis endpoint cannot block the caller beyond 3 seconds.
- **Exception surface:** catches `redis.exceptions.TimeoutError`, `redis.exceptions.ConnectionError`, and the broader `redis.RedisError` base class (which also covers auth/protocol errors), so no Redis-specific exception can propagate up and crash the caller.
- **Fail state:** on any caught exception, `self._client` is explicitly set to `None` rather than left in a partially-initialized state.
- **Self-healing read:** the `redis` property re-attempts `_init_redis()` lazily whenever `_client` is `None` and the property is accessed — meaning a worker registry that started with Redis unavailable will silently reconnect on the next call if Redis comes back up, with no restart required.
- **Every public method** (`set_engine_task_arn`, `get_engine_task_arn`, `delete_engine_task_arn`, and the generator equivalents) checks `if not self.redis`: first and returns a safe default (`False` for writes, `None` for reads) instead of raising — callers never need to wrap these in their own try/except.
- `app/core/config.py` defaults `REDIS_URL` / `REDIS_HOST` / `REDIS_PASSWORD` to `None` in the active (non-commented) block, rather than embedding a literal cluster hostname — evidence that credential-in-source-code was previously an issue in this repo and was deliberately hardened.

3.3 Redis Circuit Breaker — Alert Engine Sliding Windows

`AlertEngineService` (`app/services/alert_engine.py`) implements an **independent** copy of the same defensive pattern for its own Redis usage — it does not share a client with `WorkerRegistry`. Its Redis dependency backs sliding-window event counting for alert threshold evaluation, using a `metric:{metric_id}` **ZSET** keyed by event timestamp.

```
# app/services/alert_engine.py (AlertEngineService)
def _init_redis(self):
    """Initialize Redis connection with strict timeouts."""
    if not settings.REDIS_HOST:
        print("[AlertEngine] REDIS_HOST not set. Falling back to SQLite
for metric windows.")
        self._redis_available = False
        self._redis_client = None
        return

    try:
        # Phase 6: Initialize with strict timeouts to prevent infinite
hanging
        self._redis_client = redis.Redis(
            host=settings.REDIS_HOST,
            port=settings.REDIS_PORT,
            password=settings.REDIS_PASSWORD,
            db=settings.REDIS_DB,
            decode_responses=True,
            socket_timeout=3.0,          # Fails if an operation takes >
3s
            socket_connect_timeout=3.0  # Fails if connection takes >
3s
        )
        self._redis_client.ping()
        self._redis_available = True
        print(f"[AlertEngine] Connected to Redis:
{settings.REDIS_HOST}")
    except (redis.exceptions.TimeoutError,
redis.exceptions.ConnectionError, redis.RedisError) as e:
        print(f"[AlertEngine] Redis timeout/connection error: {e}")
        print("[AlertEngine] Falling back to SQLite for metric windows")
        self._redis_available = False
        self._redis_client = None
```

Unlike `WorkerRegistry`, which gates on `REDIS_URL`, this service gates on `settings.REDIS_HOST` directly (constructing the client from discrete `host` / `port` / `password` / `db` fields rather than a single DSN string) — a source-confirmed inconsistency between the two Redis clients worth flattening in a future refactor, but not a functional bug since both ultimately resolve to the same target.

Every windowing operation is dual-pathed — Redis first, SQLite second — with live demotion on failure, not just a fixed startup check:

```
def _get_metric_key(self, metric_id: int) -> str:
    return f"metric:{metric_id}"

def update_metric_window(self, metric_id: int, event_timestamp: str,
                        window_sec: int):
    """Update the sliding window for a metric using Redis ZSET."""
    if self._redis_available and self.redis:
        try:
            self._update_metric_window_redis(metric_id, event_timestamp,
                                             window_sec)
            return
        except (redis.exceptions.TimeoutError,
                redis.exceptions.ConnectionError, redis.RedisError) as e:
            print(f"[AlertEngine] Redis timeout/error, falling back to
SQLite: {e}")
            self._redis_available = False

    self._update_metric_window_sqlite(metric_id, event_timestamp,
                                     window_sec)

def _update_metric_window_redis(self, metric_id: int, event_timestamp:
                               str, window_sec: int):
    """Update sliding window using Redis."""
    key = self._get_metric_key(metric_id)

    try:
        if isinstance(event_timestamp, str):
            dt = datetime.fromisoformat(event_timestamp.replace('Z',
'+00:00'))
        else:
            dt = event_timestamp
        timestamp_score = dt.timestamp()
    except:
        timestamp_score = time.time()

    member = f"{timestamp_score}:{time.time_ns()}"

    self.redis.zadd(key, {member: timestamp_score})
    window_start = time.time() - window_sec
    self.redis.zremrangebyscore(key, 0, window_start)
    self.redis.expire(key, window_sec + 60)

def _update_metric_window_sqlite(self, metric_id: int, event_timestamp:
                                str, window_sec: int):
    """Fallback: Update sliding window using SQLite."""
    sql = """
        INSERT INTO metric_windows (metric_id, event_timestamp)
```

```

        VALUES (:metric_id, :event_timestamp)
    """
    self.execute(sql, {"metric_id": metric_id, "event_timestamp":
event_timestamp})

    window_start = (datetime.utcnow() -
timedelta(seconds=window_sec)).isoformat()
    sql = """
        DELETE FROM metric_windows
        WHERE metric_id = :metric_id AND event_timestamp < :window_start
    """
    self.execute(sql, {"metric_id": metric_id, "window_start":
window_start})

def get_window_count(self, metric_id: int, window_sec: int) -> int:
    """Get the count of events in the current window for a metric."""
    if self._redis_available and self.redis:
        try:
            return self._get_window_count_redis(metric_id, window_sec)
        except (redis.exceptions.TimeoutError,
redis.exceptions.ConnectionError, redis.RedisError) as e:
            print(f"[AlertEngine] Redis timeout/error, falling back to
SQLite: {e}")
            self._redis_available = False

    return self._get_window_count_sqlite(metric_id, window_sec)

def _get_window_count_redis(self, metric_id: int, window_sec: int) ->
int:
    """Get window count using Redis."""
    key = self._get_metric_key(metric_id)
    now = time.time()
    window_start = now - window_sec
    return self.redis.zcount(key, window_start, now)

def _get_window_count_sqlite(self, metric_id: int, window_sec: int) ->
int:
    """Fallback: Get window count using SQLite."""
    window_start = (datetime.utcnow() -
timedelta(seconds=window_sec)).isoformat()
    sql = """
        SELECT COUNT(*) as count
        FROM metric_windows
        WHERE metric_id = :metric_id AND event_timestamp >=
:window_start
    """
    result = self.execute(sql, {"metric_id": metric_id, "window_start":
window_start})
    return result[0]["count"] if result else 0

```

Key resilience properties confirmed in source:

- **ZSET member encoding** is `f"{timestamp_score}:{time.time_ns()}"` — the nanosecond suffix guarantees member uniqueness even when two events land in the same wall-clock second, which a plain timestamp-as-member scheme would silently collide/overwrite.
- **Self-trimming window**: every write both adds the new member (`zadd`) and evicts everything older than the window (`zremrangebyscore(key, 0, window_start)`) in the same call — the ZSET never grows unbounded.
- **TTL safety net**: `self.redis.expire(key, window_sec + 60)` — even if a metric stops receiving events entirely, its ZSET key self-destructs `window_sec + 60` seconds after the last write, so dead metrics don't leak Redis memory indefinitely.
- **Live demotion, not just startup gating**: `_redis_available` is flipped to `False` inside the exception handler of both `update_metric_window` and `get_window_count`, not only in `_init_redis`. This means a Redis instance that dies *mid-session* (after a successful startup ping) still triggers fallback on the very next operation, without waiting for a restart.
- **Symmetric SQLite fallback**: `metric_windows` is a literal SQLite table (`metric_id`, `event_timestamp`) that mirrors the ZSET's data model — insert-then-prune on write, range-count on read — so alert evaluation produces the same semantic result (count of events within `window_sec`) regardless of which backend is active.
- The same live-demotion pattern is applied to `get_last_processed_id` / `save_last_processed_id` (Redis key `alert_engine:last_processed_id`, in-memory `self._last_processed_id` as the fallback) so the event-polling cursor survives a Redis outage without reprocessing or dropping events.

Correction against reference context: the reference material characterizes this as simply "independent Redis fallback for sliding-window metric tracking via ZSETs," implying Redis-or-nothing. Source shows a third state the reference omits: SQLite is a **fully symmetric, permanent fallback backend** (via the `metric_windows` table), not a placeholder — alert evaluation continues functioning indefinitely on SQLite alone if Redis never recovers, with no feature loss beyond raw throughput.

Phase 4: AI Orchestration, Agents & Memory Management

4.1 LLM Provider Selection (`app/services/llm.py`)

`LLMService` is provider-agnostic and resolves exactly one active backend **once, at import time**, by checking API keys in strict priority order: **Qubrid** → **OpenAI** → **Gemini**. The first populated key wins; lower-priority providers are never even instantiated.

```
# app/services/llm.py
import time
```

```

import asyncio
from app.core.config import settings
from app.core.logger import logger

try:
    import openai
except ImportError:
    openai = None

try:
    import google.generativeai as genai
except ImportError:
    genai = None

class LLMService:
    def __init__(self):
        # Priority: Qubrid > OpenAI > Gemini
        if getattr(settings, "QUBRID_API_KEY", None):
            if openai is None:
                raise ImportError("openai package is required for Qubrid
API usage.")
            self.client = openai.AsyncOpenAI(
                api_key=settings.QUBRID_API_KEY,
                base_url=settings.QUBRID_BASE_URL,
                timeout=20.0
            )
            self.provider = "qubrid"
            self.model_name = getattr(settings, "QUBRID_MODEL_NAME",
"meta-llama/llama-3.3-70B-Instruct")
            logger.info(f"LLM Service initialized with Qubrid provider,
model: {self.model_name}")
        elif getattr(settings, "OPENAI_API_KEY", None):
            if openai is None:
                raise ImportError("openai package is required for
OPENAI_API_KEY usage.")
            self.client = openai.AsyncOpenAI(
                api_key=settings.OPENAI_API_KEY,
                base_url=settings.OPENAI_BASE_URL,
                timeout=20.0
            )
            self.provider = "openai"
            self.model_name = getattr(settings, "OPENAI_MODEL_NAME",
"gpt-4o-mini")
            logger.info(f"LLM Service initialized with OpenAI provider,
model: {self.model_name}")
        elif getattr(settings, "GEMINI_API_KEY", None):
            if genai is None:
                raise ImportError("google-generativeai package is
required for GEMINI_API_KEY usage.")
            genai.configure(api_key=settings.GEMINI_API_KEY)

```

```

        self.provider = "gemini"
        self.model_name = getattr(settings, "GEMINI_MODEL_NAME",
"gemini-2.5-flash-lite")
        safety_settings = [
            {"category": "HARM_CATEGORY_HARASSMENT", "threshold":
"BLOCK_NONE"},
            {"category": "HARM_CATEGORY_HATE_SPEECH", "threshold":
"BLOCK_NONE"},
            {"category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",
"threshold": "BLOCK_NONE"},
            {"category": "HARM_CATEGORY_DANGEROUS_CONTENT",
"threshold": "BLOCK_NONE"},
        ]
        self.model = genai.GenerativeModel(
            model_name=self.model_name,
            safety_settings=safety_settings
        )
        logger.info(f"LLM Service initialized with Gemini provider,
model: {self.model_name}")
    else:
        print("Warning: No LLM API key found in settings.")
        self.provider = None
        self.model = None

    async def _call_openai_compatible(self, prompt: str, model_override:
str = None) -> str:
        """Call OpenAI-compatible ChatCompletion (works for OpenAI,
Ollama, Qubrid, etc.)."""
        response = await self.client.chat.completions.create(
            model=model_override or self.model_name,
            messages=[{"role": "user", "content": prompt}],
            temperature=0.0,
            stream=False,
        )
        return response.choices[0].message.content

    async def generate_response(self, prompt: str, model_name: str =
None) -> str:
        if not self.provider:
            logger.error("LLM Service not configured (no API key
found).")
            return "LLM Service not configured."

        selected_model = model_name or self.model_name
        logger.info(f"LLM [{self.provider}] using model
[{selected_model}] generating response...")

        attempts = 0
        while attempts < 5:
            try:
                if self.provider in ("openai", "qubrid"):

```

```

        res = await self._call_openai_compatible(prompt,
model_override=selected_model)
    else: # gemini
        model_to_use = self.model
        if model_name:
            model_to_use = genai.GenerativeModel(
                model_name=model_name,
                safety_settings=[
                    {"category": "HARM_CATEGORY_HARASSMENT",
"threshold": "BLOCK_NONE"},
                    {"category":
"HARM_CATEGORY_HATE_SPEECH", "threshold": "BLOCK_NONE"},
                    {"category":
"HARM_CATEGORY_SEXUALLY_EXPLICIT", "threshold": "BLOCK_NONE"},
                    {"category":
"HARM_CATEGORY_DANGEROUS_CONTENT", "threshold": "BLOCK_NONE"},
                ]
            )

        response = model_to_use.generate_content(prompt)

        if not response.candidates or
response.candidates[0].finish_reason != 1:
            if response.candidates and
response.candidates[0].finish_reason == 3:
                logger.warning(f"LLM response blocked by
safety filters for model {selected_model}.")
                return "Error: Response blocked by safety
filters."

        res = response.text

        logger.info(f"LLM response received from
[{selected_model}]. Snippet: {res[:50]}...")
        return res
    except Exception as e:
        err_msg = str(e).lower()
        if "429" in err_msg or "rate limit" in err_msg or
"quota" in err_msg or "503" in err_msg or "resourceexhausted" in err_msg
or "limit reached" in err_msg:
            attempts += 1
            wait = 2 ** attempts
            logger.warning(f"LLM rate limit / resource
exhaustion encountered, retrying in {wait}s (attempt {attempts})")
            await asyncio.sleep(wait)
            continue
        logger.error(f"LLM Error: {str(e)}")
        return f"Error generating response: {str(e)}"
    logger.error("LLM service rate limit exceeded after multiple
retries.")
    return "Error: LLM service rate limit exceeded after multiple

```

```
retries."

llm_service = LLMService()
```

Confirmed operational details:

- **Deterministic SQL generation:** the OpenAI-compatible path (used for both `openai` and `qubrid` providers) hardcodes `temperature=0.0` in `_call_openai_compatible`, ensuring NL2SQL generation is reproducible — this is distinct from and lower-variance than `SentinelBrainstormer`'s `temperature=0.8` (§4.2).
- **Gemini safety posture:** all four harm categories (`HARASSMENT`, `HATE_SPEECH`, `SEXUALLY_EXPLICIT`, `DANGEROUS_CONTENT`) are explicitly set to `BLOCK_NONE` — this is a deliberate choice to avoid the model refusing to summarize legitimate but sensitive-sounding financial-crime/fraud/compliance query results (e.g. "money laundering", "account takeover") that are core to this platform's domain.
- **Finish-reason handling:** `finish_reason 1 = STOP` (normal completion); `finish_reason 3 = SAFETY` (blocked) — only the safety-block case is special-cased with a user-facing error string; any other non-`STOP` finish reason falls through silently to `response.text`, which will raise if no valid candidate exists.
- **Retry policy:** up to 5 attempts, exponential backoff (`2 ** attempts` seconds: 2, 4, 8, 16, 32), triggered by substring-matching the lowercased exception message against `429`, `rate limit`, `quota`, `503`, `resourceexhausted`, `limit reached` — a deliberately broad, provider-agnostic match since Gemini's and OpenAI's/Qubrid's SDKs raise different exception types for the same underlying condition.
- **Model default divergence:** `config.py`'s `GEMINI_MODEL_NAME` default is `"gemini-3-flash-preview"`, but `llm.py`'s own inline fallback (`getattr(settings, "GEMINI_MODEL_NAME", "gemini-2.5-flash-lite")`) is only ever reached if the `settings` object is missing the attribute entirely — which it isn't, since `config.py` always defines it. In practice, the effective default model is always whatever `config.py` specifies.

Architectural inconsistency confirmed in source: `SentinelBrainstormer` (§4.2) does **not** route through `LLMService` / `llm_service` at all. It constructs its own `langchain_openai.ChatOpenAI` client directly against `settings.OPENAI_API_KEY` / `settings.OPENAI_MODEL_NAME` / `settings.OPENAI_BASE_URL` — completely bypassing the Qubrid → OpenAI → Gemini priority chain above. If only `GEMINI_API_KEY` is populated (Gemini being the active provider for the rest of the platform), the Sentinel brainstorming path has no credentials and will fail at the LangChain client layer. This is a structural gap, not a matter of configuration — fixing it requires either routing `SentinelBrainstormer` through `llm_service`, or giving it its own explicit provider-fallback logic.

4.2 SentinelBrainstormer (app/orchestration/sentinel_agent.py)

Autonomous mission-generation agent that produces its own audit questions ("Deep Audit Missions") across weighted business domains, without a human supplying the query.

```
# app/orchestration/sentinel_agent.py
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from app.core.config import settings
from app.orchestration.scan_memory import scan_memory
import json

class SentinelBrainstormer:
    def __init__(self):
        self.llm = ChatOpenAI(
            model=settings.OPENAI_MODEL_NAME,
            api_key=settings.OPENAI_API_KEY,
            base_url=settings.OPENAI_BASE_URL,
            temperature=0.8
        )

        self.schema_context = """
        ACTUAL TABLES:
        - users (user_id, email, full_name, country, phone,
        date_of_birth, kyc_status, kyc_verified_at, kyc_expiry_date, risk_level,
        risk_score, is_pep, account_status, created_at, updated_at)
        - transactions (txn_id, user_id, txn_type, instrument, amount,
        currency, amount_usd, status, flag_reason, payment_method, external_ref,
        ip_address, created_at, processed_at)
        - login_events (event_id, user_id, email_attempted, ip_address,
        country, city, device_type, device_fingerprint, user_agent, status,
        failure_reason, created_at)
        """

        async def brainstorm_missions(self, count_per_domain=2):
            adaptive = scan_memory.get_adaptive_context()

            if settings.ADAPTIVE_ENABLED and adaptive["scan_count"] > 0:
                domain_weights = adaptive["domain_weights"]
            else:
                domain_weights = {"security": count_per_domain,
                "compliance": count_per_domain,
                "risk": count_per_domain, "operations":
                count_per_domain}

            total_count = sum(domain_weights.values())

            prompt = ChatPromptTemplate.from_template("""
            You are the "Sentinel Brain", an autonomous security and
```

compliance agent for a financial platform.

Your task is to brainstorm dynamic, high-impact "Deep Audit Missions" based on the database schema.

SCHEMA:
{schema}

MISSION ALLOCATION BY DOMAIN:

- security: {sec_count} missions (threats, fraud, unusual logins, account takeovers)
- compliance: {comp_count} missions (regulatory violations, KYC gaps, PEP monitoring, AML)
- operations: {ops_count} missions (payment failures, system health, user performance)
- risk: {risk_count} missions (high-risk exposure, portfolio imbalances)

SCAN INTELLIGENCE (from {scan_count} previous scans):

FOCUS AREAS (generate DEEPER missions on these – they were critical last scan):
{focus_areas}

QUERIES TO AVOID (already explored recently – generate NEW angles):
{avoid_queries}

GOAL:
Generate exactly {total_count} distinct audit questions (missions).
The questions should be sophisticated, seeking hidden patterns or critical risks.
DIVERSIFY: Do NOT repeat previous queries. Explore new angles each scan.

RESPONSE FORMAT:

Provide ONLY a JSON list of objects:

```
[
  {
    "id": "unique_string",
    "name": "Short Mission Name",
    "query": "The natural language question for the AI to solve",
    "domain": "one of the domains above",
    "severity": "CRITICAL, HIGH, or MEDIUM"
  }
]
```

```
chain = prompt | self.llm
response = await chain.ainvoke({
```

```

        "schema": self.schema_context,
        "sec_count": domain_weights.get("security", 2),
        "comp_count": domain_weights.get("compliance", 2),
        "ops_count": domain_weights.get("operations", 2),
        "risk_count": domain_weights.get("risk", 2),
        "total_count": total_count,
        "scan_count": adaptive["scan_count"],
        "focus_areas": "\n".join(adaptive["focus_areas"]) or "No
previous data. This is the first scan.",
        "avoid_queries": "\n".join(f"- {q}" for q in
adaptive["avoid_queries"]) or "None – first scan."
    })

    try:
        text = response.content
        if "```json" in text:
            text = text.split("```json")[1].split("```")[0].strip()
        elif "```" in text:
            text = text.split("```")[1].split("```")[0].strip()

        return json.loads(text)
    except Exception as e:
        print(f"Failed to parse brainstormed missions: {e}")
        return []

```

Confirmed behavior:

- **Mission diversity via temperature:** `temperature=0.8` on the `ChatOpenAI` constructor – deliberately high-variance versus the `temperature=0.0` used elsewhere in the pipeline for deterministic SQL generation (§4.1), because this agent's entire job is to generate *novel* audit angles rather than the single "correct" answer.
- **Adaptive vs. static allocation:** if `settings.ADAPTIVE_ENABLED` is `True` and there is at least one prior recorded scan (`adaptive["scan_count"] > 0`), domain mission counts come from `scan_memory.get_adaptive_context()["domain_weights"]` (§4.3). Otherwise it falls back to a flat, static split of `count_per_domain` (default 2) across exactly four domains: `security`, `compliance`, `risk`, `operations`. Note this static fallback branch does **not** include `retail_clothing`, even though the adaptive weighting formula in `scan_memory.py` computes a weight for it – `retail_clothing` missions are only generated once real adaptive history exists.
- **Context injection, not fine-tuning:** "learning" from prior scans is entirely prompt-based – `focus_areas` (critical findings from the immediately preceding scan) and `avoid_queries` (up to the 15 most recent distinct queries across the last 3 scans) are interpolated as plain text into the prompt template; there is no model weight update or embedding-based retrieval involved.
- **Structured-output parsing is defensive but not schema-validated:** the LLM is instructed to return "ONLY a JSON list," and the parser strips Markdown code fences

(`'''json` or plain `'''`) before calling `json.loads`. Any parse failure (malformed JSON, extra prose despite instructions) is caught and logged, returning an empty list rather than raising — a brainstorming failure degrades to "zero missions generated this cycle," not a crash.

- **Schema context is hardcoded, not introspected:** `self.schema_context` is a literal triple-quoted string describing three tables (`users`, `transactions`, `login_events`) — it is not generated by inspecting the live database schema at runtime, so if the underlying schema changes, this string must be updated manually.

4.3 SessionScopedScanMemory (`app/orchestration/scan_memory.py`)

Per-session adaptive memory for the Sentinel agent, providing genuine filesystem-level tenant isolation (one JSON file per session) plus a separate, longer-lived full-scan archive for the history viewer UI.

```
# app/orchestration/scan_memory.py
import json
import os
import uuid
import time
import glob as globmod
from datetime import datetime
from collections import Counter

SESSION_STORE_DIR = "scan_memory_sessions"
SESSION_TTL_SECONDS = 60 * 60 * 6 # 6 hours
SCAN_HISTORY_DIR = "scan_history"

class SessionScopedScanMemory:
    """
    One instance per session_id. Each session's history lives in its own
    file, keyed by a UUID, so no tenant can read or influence another's
    adaptive context.
    """

    MAX_HISTORY = 50

    def __init__(self, session_id: str | None = None):
        self.session_id = session_id or str(uuid.uuid4())
        os.makedirs(SESSION_STORE_DIR, exist_ok=True)
        self._path = os.path.join(SESSION_STORE_DIR, f"
{self.session_id}.json")
        self.history = self._load()

    def _load(self) -> list:
```

```

        if os.path.exists(self._path):
            try:
                with open(self._path, "r") as f:
                    payload = json.load(f)
                    if time.time() - payload.get("_last_touched", 0) >
SESSION_TTL_SECONDS:
                        return []
                    return payload.get("history", [])
            except (json.JSONDecodeError, OSError):
                return []
        return []

    def _save(self) -> None:
        payload = {"_last_touched": time.time(), "history":
self.history[-self.MAX_HISTORY:]}
        with open(self._path, "w") as f:
            json.dump(payload, f, indent=2, default=str)

    def record_scan(self, detections: list) -> None:
        scan_record = {"timestamp": datetime.utcnow().isoformat(),
"missions": [], "domain_scores": {}, "critical_domains": []}
        domain_scores: dict[str, list[int]] = {}
        for det in detections:
            domain = det.get("domain", "general")
            risk_score = det.get("risk_score", 0)
            scan_record["missions"].append({
                "name": det.get("mission_name"), "query":
det.get("original_query", ""),
                "domain": domain, "risk_score": risk_score,
                "severity": det.get("severity"), "data_count":
det.get("data_count", 0),
            })
            domain_scores.setdefault(domain, []).append(risk_score)
            if det.get("severity") == "CRITICAL" and domain not in
scan_record["critical_domains"]:
                scan_record["critical_domains"].append(domain)
            scan_record["domain_scores"] = {d: round(sum(s) / len(s)) for d,
s in domain_scores.items()}
        self.history.append(scan_record)
        self._save()

    @staticmethod
    def evict_expired_sessions() -> int:
        if not os.path.exists(SESSION_STORE_DIR):
            return 0
        evicted = 0
        cutoff = time.time() - SESSION_TTL_SECONDS
        for fname in os.listdir(SESSION_STORE_DIR):
            fpath = os.path.join(SESSION_STORE_DIR, fname)
            try:
                with open(fpath) as f:

```

```

        if json.load(f).get("_last_touched", 0) < cutoff:
            os.remove(fpath)
            evicted += 1
        except (json.JSONDecodeError, OSError):
            os.remove(fpath)
            evicted += 1
    return evicted

def get_adaptive_context(self) -> dict:
    if not self.history:
        return {
            "domain_weights": {"security": 2, "compliance": 2,
"risk": 2, "operations": 2},
            "avoid_queries": [],
            "focus_areas": [],
            "scan_count": 0,
        }

    recent = self.history[-5:]
    domain_avg_scores = Counter()
    domain_counts = Counter()
    for scan in recent:
        for d, score in scan.get("domain_scores", {}).items():
            domain_avg_scores[d] += score
            domain_counts[d] += 1

    domain_weights = {}
    for domain in ["security", "compliance", "risk", "operations",
"retail_clothing"]:
        avg = domain_avg_scores.get(domain, 0) /
max(domain_counts.get(domain, 1), 1)
        domain_weights[domain] = min(4, max(1, round(2 + avg / 40)))

    recent_queries = []
    for scan in recent[-3:]:
        for m in scan.get("missions", []):
            q = m.get("query", "")
            if q and q not in recent_queries:
                recent_queries.append(q)

    focus_areas = []
    if recent:
        last_scan = recent[-1]
        for m in last_scan.get("missions", []):
            if m.get("severity") == "CRITICAL":
                focus_areas.append(
                    f"CRITICAL finding in {m['domain']}:
'{m['name']}' "
                    f"(risk score {m['risk_score']}). Generate
deeper variations."
                )

```

```

        return {
            "domain_weights": domain_weights,
            "avoid_queries": recent_queries[-15:],
            "focus_areas": focus_areas,
            "scan_count": len(self.history),
        }

# — Full Scan Persistence (for history viewer)


---



def save_full_scan(self, scan_id: str, detections: list, clusters:
list = None, narrative: dict = None):
    os.makedirs(SCAN_HISTORY_DIR, exist_ok=True)
    critical_count = sum(1 for d in detections if d.get("severity")
== "CRITICAL")
    high_count = sum(1 for d in detections if d.get("severity") ==
"HIGH")
    top_scores = sorted([d.get("risk_score", 0) for d in
detections], reverse=True)
    overall_risk = round(sum(top_scores[:3]) /
max(len(top_scores[:3]), 1))

    if narrative and narrative.get("overall_risk"):
        overall_risk = narrative["overall_risk"]

    overall_severity = "LOW"
    if critical_count > 0:
        overall_severity = "CRITICAL"
    elif high_count > 0:
        overall_severity = "HIGH"

    data = {
        "scan_id": scan_id,
        "timestamp": datetime.utcnow().isoformat(),
        "stats": {
            "total_missions": len(detections),
            "critical_count": critical_count,
            "high_count": high_count,
            "overall_risk": overall_risk,
            "overall_severity": overall_severity,
        },
        "detections": detections,
        "clusters": clusters or [],
        "narrative": narrative,
    }

    filepath = os.path.join(SCAN_HISTORY_DIR, f"{scan_id}.json")
    try:
        with open(filepath, "w") as f:
            json.dump(data, f, indent=2, default=str)

```

```

except Exception as e:
    print(f"Failed to save full scan: {e}")

self._prune_old_scans()

def _prune_old_scans(self):
    files = sorted(globmod.glob(os.path.join(SCAN_HISTORY_DIR,
"scan-*.json")))
    if len(files) > self.MAX_HISTORY:
        for old in files[: len(files) - self.MAX_HISTORY]:
            try:
                os.remove(old)
            except Exception:
                pass

def list_scans(self) -> list:
    if not os.path.isdir(SCAN_HISTORY_DIR):
        return []
    scans = []
    for filepath in globmod.glob(os.path.join(SCAN_HISTORY_DIR,
"scan-*.json")):
        try:
            with open(filepath, "r") as f:
                data = json.load(f)
            scans.append({
                "scan_id": data["scan_id"],
                "timestamp": data["timestamp"],
                **data.get("stats", {}),
            })
        except Exception:
            continue
    scans.sort(key=lambda s: s["timestamp"], reverse=True)
    return scans

def get_scan(self, scan_id: str) -> dict | None:
    filepath = os.path.join(SCAN_HISTORY_DIR, f"{scan_id}.json")
    if not os.path.exists(filepath):
        return None
    try:
        with open(filepath, "r") as f:
            return json.load(f)
    except Exception:
        return None

```

Confirmed mechanics, with precise semantics called out:

- **Filesystem-level tenant isolation:** each `SessionScopedScanMemory` instance owns exactly one file, `scan_memory_sessions/<session_id>.json`, where `session_id` is a UUID4 (auto-generated if not supplied). Two sessions cannot read or influence

each other's adaptive context — there is no shared in-process dict keyed by session ID that a bug could cross-contaminate.

- **TTL is checked lazily on read, not enforced by a background timer.** `_load()` compares `time.time() - payload["_last_touched"]` against `SESSION_TTL_SECONDS` (`60 * 60 * 6 = 21,600 seconds / 6 hours`). If the session file is older than the TTL, `_load()` returns an **empty list** — the history is treated as expired **in-memory**, but the physical JSON file on disk is left untouched at this point.
- **Actual file deletion happens only via `evict_expired_sessions()`**, a `@staticmethod` that must be invoked explicitly (e.g. on a schedule, at startup, or via an admin endpoint — it is not a background thread and does not run automatically). It walks every file in `scan_memory_sessions/`, deletes any whose `_last_touched` predates `time.time() - SESSION_TTL_SECONDS`, and additionally deletes (and counts as evicted) any file that fails to parse as JSON — self-healing against corrupted session files.
- **Five-domain adaptive weighting**, not four: `security`, `compliance`, `risk`, `operations`, and `retail_clothing`. The weight formula is:

```
avg = domain_avg_scores.get(domain, 0) /  
max(domain_counts.get(domain, 1), 1)  
weight = min(4, max(1, round(2 + avg / 40)))
```

This computes, per domain, the average risk score across up to the last 5 recorded scans (`recent = self.history[-5:]`), then maps that average through `2 + avg/40`, clamped to the integer range **[1, 4]**. A domain with average risk score 0 yields `round(2 + 0) = 2` (baseline); a domain averaging 80+ saturates at the maximum weight of 4; a domain that has never appeared in `domain_scores` defaults its average to 0 (via `Counter`'s zero-default and the `max(..., 1)` divide-by-zero guard) and also resolves to weight 2.

- **`retail_clothing` has supporting infrastructure beyond this formula** — a dedicated domain config file at `app/data/domains/retail_clothing.json` and a corresponding prompt block in `app/modules/preprocessing/assets/domain_config.py` — confirming it is a real, intentionally-built fifth domain rather than a leftover or placeholder string.
- **Cold-start default is only four domains:** `get_adaptive_context()`'s early-return branch (`if not self.history`) hardcodes `{"security": 2, "compliance": 2, "risk": 2, "operations": 2}` with no `retail_clothing` key at all — consistent with `SentinelBrainstormer`'s own static fallback (§4.2). `retail_clothing` missions can only be generated once at least one scan has been recorded and the full weighting branch executes.
- **`avoid_queries` is deduplicated** (`if q and q not in recent_queries`) across the missions of the last 3 scans (`recent[-3:]`), then capped to the most recent 15

```
entries ( recent_queries[-15:] ).
```

- **focus_areas** is derived only from the single most recent scan (`recent[-1]`), not `recent[-5:]`) and only includes missions with `severity == "CRITICAL"` — `HIGH / MEDIUM` findings never generate a focus-area prompt hint, by design.
- **Dual persistence layers serve different purposes:** `history` (in the session JSON, capped at `MAX_HISTORY = 50` scans, subject to the 6-hour TTL) drives the *adaptive prompting* described above. `save_full_scan()` writes a **separate, complete** record — full `detections`, `clusters`, and `narrative` payloads plus aggregate `stats` (mission/critical/high counts, computed `overall_risk` as the mean of the top 3 risk scores unless a narrative-supplied `overall_risk` overrides it, and derived `overall_severity`) — to `scan_history/<scan_id>.json`, independently capped at 50 files via `_prune_old_scans()` with oldest-first deletion by filename sort order. This second store has **no TTL of its own** and powers the scan-history viewer UI (`list_scans()` , `get_scan()`) independently of session adaptive memory.

4.4 NL2SQL Validation (`app/modules/validation.py`)

```
# app/modules/validation.py
import sqlglot
from typing import Optional, Tuple

class ValidationModule:
    """
    HLD 3.5: Validation and Repair Module
    Performs multi-level checks on generated SQL.
    """

    def validate(self, sql: str) -> Tuple[bool, Optional[str]]:
        """
        Validates SQL syntax using sqlglot.
        Returns (is_valid, error_message)
        """
        try:
            sqlglot.transpile(sql)
            # Further checks: Schema conformance (mocked here)
            if "DROP" in sql.upper():
                return False, "DROP statements are not allowed."
            return True, None
        except Exception as e:
            return False, str(e)

validation_module = ValidationModule()
```

Confirmed scope and limitations:

- `sqlglot.transpile(sql)` is used purely as a syntax/parseability check. It is called with no `read=` / `write=` dialect arguments, so it uses `sqlglot`'s default dialect on both ends — this validates that the SQL is *well-formed*, not that it conforms to any specific target database's dialect quirks. A `sqlglot.errors.ParseError` (or any other exception raised during transpilation) is caught by the bare `except` Exception as `e` and converted into a `(False, str(e))` validation failure rather than propagating.
- The safety rule is a single case-insensitive substring check: `"DROP" in sql.upper()`. This is confirmed to be **exactly that** — a naive substring match, not an AST-level statement-type check. Practical implications worth documenting explicitly for anyone extending this module:
 - It **will** false-positive-block a syntactically valid, harmless query that merely contains the substring "DROP" outside of DDL context — e.g. a column or literal string value named `drop_reason`, or a `WHERE` clause value like `'dropped'`.
 - It **cannot** be trivially bypassed by whitespace/comment obfuscation of the keyword itself (since it's a raw uppercase substring check against the whole string, not a tokenizer), but it also provides **no protection against other destructive statement types** — `DELETE`, `TRUNCATE`, `ALTER`, `UPDATE` without a `WHERE` clause, etc. are all syntactically valid and will pass `validate()` unmodified.
 - The inline comment `# Further checks: Schema conformance (mocked here)` is a direct admission in source that schema-aware validation (confirming referenced tables/columns actually exist, type-checking predicates, etc.) is **not implemented** — only syntax validity and the single DROP-substring rule are enforced today.
- No connection to the LLM-generation retry loop is present in this file — `ValidationModule.validate()` is a pure function of the SQL string; whether/how a validation failure triggers a regeneration attempt upstream in the NL2SQL pipeline is outside this module's scope and is not shown here.

Correction against reference context: the reference material describes this as "SQL Validation: `sqlglot` transpiles/parses SQL; hard check blocks DROP statements," which is accurate as far as it goes but could be read as implying broader statement-type coverage than exists. For engineering purposes, treat this validator as a **syntax gate + single-keyword denylist**, not a general SQL-injection or destructive-statement firewall — any hardening work (blocking `DELETE` / `TRUNCATE` / unqualified `UPDATE`, enforcing a read-only statement allowlist via `sqlglot`'s AST rather than string matching) is a real, unimplemented gap, not a documentation omission.